PYCON HK 2026

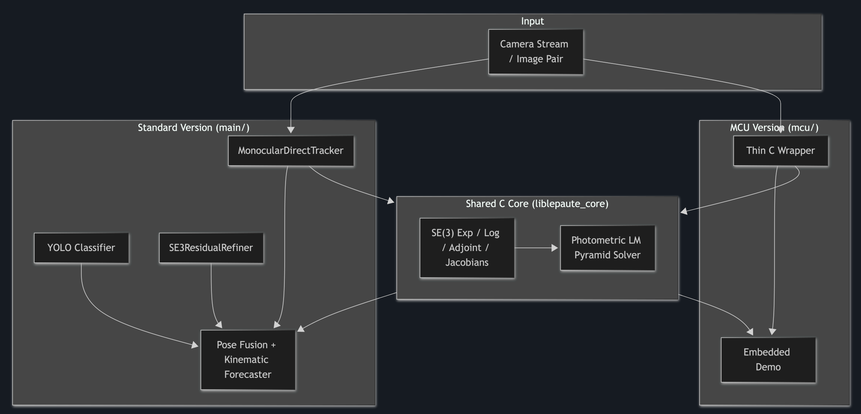
```
// Built with Python 3.13.12 & PyTorch 2.14.1
// Core architecture by  Carson Wu (@dev1virtuoso)
```

# Beyond Brute-Force: Embedding Lie Group Theory into PyTorch Layers for Spatial Perception

L.E.P.A.U.T.E. Framework — Making 3D Spatial Computing Fast & Smooth in PyTorch

Developed by Carson Wu (@dev1virtuoso)

## System Architecture & Pipeline Flow




## Problem & Motivation: Why Lie Groups for Robust Spatial Perception?

**The Problem:** Autonomous driving and FSD-style 3D spatial perception require robust reasoning about continuous rigid-body transformations (3D rotation, translation, depth, and scale). Standard CNNs treat images as flat 2D pixel grids without built-in geometric constraints. As a result, gradients distort rotation matrices, Euler angles suffer from discontinuities and gimbal lock, and models must implicitly rediscover physical laws from scratch using massive datasets.

**Why Lie Groups?** We chose a **Lie Group ("SE"(3)) approach** because it provides the natural mathematical language for continuous rigid motions. By operating on smooth manifolds and their flat 6D tangent spaces ($\mathfrak{se}(3)$), all transformations remain strictly valid by design. This injects rigorous geometric equivariance ($f(g \cdot x) = g \cdot f(x)$) directly into PyTorch layers, enabling networks to learn physical residuals rather than raw geometry.

geometry.py — Lie Group Equivariant Exponential Mapping

Py geometry.py — Lie Group Equivariant Exponential Mapping

Mapping 6D Lie algebra tangent vectors ($\mathfrak{se}(3)$) into smooth SE(3) manifold transformation matrices via closed-form exponential maps.

```python
def se3_exp_map(xi: torch.Tensor) -> torch.Tensor::
# Extract 3D translation (rho) and 3D rotation (phi) from tangent vector
rho, phi = xi[:, :3], xi[:, 3:]
theta = torch.sqrt(torch.clamp(torch.sum(phi ** 2, dim=1), min=1e-20))
# Rodrigues closed-form formula mapping tangent twist to orthogonal SE(3) matrix
return rodrigues_exp_with_taylor_fallback(rho, phi, theta)
```

- Geometric Equivariance
- Geometric Consistency Enforced

## Asynchronous Subsystem Architecture ('main.py' & 'InferenceWorker')

**GIL Bypass via Process Isolation:** To prevent heavy neural network inference (`SE3ResidualRefiner` and `YOLOClassifier`) from stalling the real-time direct visual tracker, L.E.P.A.U.T.E. uses Python's `multiprocessing.get_context('spawn')` to isolate deep learning pipelines in a dedicated background worker process.

main.py — Multi-Process Non-Blocking Async Queue

Py main.py — Multi-Process Non-Blocking Async Queue

```python
class InferenceWorker:
def __init__(self, config, model_path):
self.ctx = mp.get_context('spawn') # Bypasses Python GIL lock
self.job_queue = self.ctx.Queue(maxsize=5)
self.result_queue = self.ctx.Queue()
self.process = self.ctx.Process(target=self.__run_process, daemon=True)
self.process.start()
```

- Zero Frame Drop
- IPC Shared Memory
- GIL Isolated

## Performance Leap & Verified Benchmarks

PIPELINE END-TO-END SPEED

**≈ 0.6 FPS → 12 FPS**

### Environment

macOS ARM64 | 12-core CPU | Python 3.13.12 | PyTorch 2.14.1

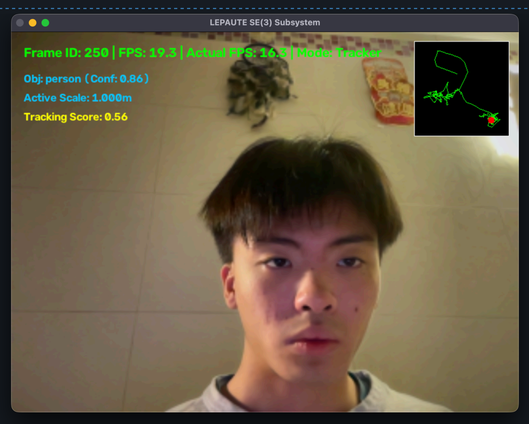
### Refiner Inference Latency

- CPU Mode: Mean 21.72 ms (≈ 46.03 FPS, Median: 21.42 ms)
- MPS GPU Mode: Mean 7.10 ms (≈ 140.82 FPS, Median: 6.29 ms)

### Stability & Precision Metrics

- Geometric MSE: 0.0025  0.0027
- Convergence Success Rate (MSE < 0.01): 100% (1.0)
- Boundary Protection: Near-zero & near-π angle errors < $10^{-12}$

## Real-time Demo & Perception Screenshots




## Manifold Geometry (geometry.py)

geometry.py — SE(3)/SO(3) Manifold Geometry

Py geometry.py — SE(3)/SO(3) Manifold Geometry

Dedicated module handling **SE(3) / SO(3) Lie group mappings** with robust boundary protection (near-zero & near-π singularities).

```python
A = (R_pi + torch.eye(3, device=T.device…).unsqueeze(0)) * 0.5
diag_A = torch.diagonal(A, dim1=-2, dim2=-1)
max_idx = torch.argmax(diag_A, dim=-1)
batch_idx = torch.arange(R_pi.shape[0], device=T.device)
v = A[batch_idx, :, max_idx]
```

→ **1. Symmetric Matrix Projection**
**2. Dominant Axis Selection via Diagonal**

Diagonal Dominance Selection: $A = \frac{R+I}{2}$

```python
v = v / torch.clamp(torch.norm(v, dim=-1, keepdim=True), min=1e-10)
sign = torch.sign(torch.sum(v * phi_raw[mask_pi], dim=-1…))
sign[sign == 0] = 1.0
v = v * sign
```

→ **Sign Resolution using Raw Skew-Symmetric Components**

```python
mask_small = (theta < 1e-4)
mask_pi = (theta > (torch.pi - 1e-3))
mask_large =  (mask_small | mask_pi)
```

→ **Dynamic Angle Stratification for Numerical Stability**

- Python 3.13.12
- PyTorch 2.14.1
- MPS / CUDA Ready

## Repository & Code Files

Scan to explore modular code files, architecture scripts, and reproducible benchmarks on GitHub.